

Language Implementation

This chapter describes the platform-independent language features that are not covered by the grammar language ([301_grammarlanguage.html](#)).

Some code examples in this chapter are given in the Xtend language, as it is much better suited for these tasks than Java. Please refer to the Xtend documentation (<https://eclipse.dev/Xtext/xtend>) for further details. For Java developers it's extremely easy to learn, as the basics are similar and you only need to understand the additional powerful concepts.

Code Generation

Once you have a language you probably want to do something with it. There are two options, you can either write an interpreter that inspects the AST and does something based on that or you translate your language to another programming language or configuration files. In this section we're going to show how to implement a code generator for an Xtext-based language.

IGenerator2

If you go with the default MWE workflow for your language and you haven't used Xbase, you are provided with a callback stub that implements IGenerator2 (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/generator/IGenerator2.java>) by extending the AbstractGenerator (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/generator/AbstractGenerator.java>) base class. It has three methods that are called from the builder infrastructure whenever a DSL file has changed or should be translated otherwise. The three parameters passed in to those method are:

- The resource to be processed
- An instance of IFileSystemAccess2 (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/generator/IFileSystemAccess2.java>)
- An instance of IGeneratorContext (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/generator/IGeneratorContext.java>)

The IFileSystemAccess2 (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/generator/IFileSystemAccess2.java>) API abstracts over the different file systems the code generator may run over. When the code generator is triggered within the incremental build infrastructure in Eclipse the underlying file system is the one provided by Eclipse, and when the code generator is executed outside Eclipse, say in a headless build, it is `java.io.File`.

A very simple implementation of a code generator for the state machine example (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.xtext.ui.examples/projects/fowlerdsl/org.eclipse.xtext.example.fowlerdsl/src/org/eclipse/xtext/example/fowlerdsl/Statemachine.xtext>) could be the following:

```
1. class StatemachineGenerator extends AbstractGenerator {  
2.  
3.     override doGenerate(Resource resource, IFileSystemAccess2 fsa, IGeneratorConte  
    xt context) {  
4.         fsa.generateFile("relative/path/AllTheStates.txt", '''  
5.             «FOR state : resource.allContents.filter(State).toIterable»  
6.             State «state.name»  
7.             «ENDFOR»  
8.         ''' )  
9.     }  
10.  
11. }
```

Output Configurations

You don't want to deal with platform or even installation dependent paths in your code generator, rather you want to be able to configure the code generator with some basic outlet roots where the different generated files should be placed under. This is what output configurations are made for.

By default every language will have a single outlet, which points to `<project-root>/src-gen/`. The files that go here are treated as fully derived and will be erased by the compiler automatically when a new file should be generated. If you need additional outlets or want to have a different default configuration, you need to implement the interface `IOutputConfigurationProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/generator/IOutputConfigurationProvider.java>). It's straightforward to understand and the default implementation gives you a good idea about how to implement it.

With this implementation you lay out the basic defaults which can be changed by users on a workspace or per project level using the preferences.

Validation

Static analysis is one of the most interesting aspects when developing a programming language. The users of your languages will be grateful if they get informative feedback as they type. In Xtext there are basically three different kinds of validation.

Automatic Validation

Some implementation aspects (e.g. the grammar, scoping) of a language have an impact on what is required for a document or semantic model to be valid. Xtext automatically takes care of this.

Lexer/Parser: Syntactical Validation

The syntactical correctness of any textual input is validated automatically by the parser. The error messages are generated by the underlying parser technology. One can use the `ISyntaxErrorMessageProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parser/antlr/ISyntaxErrorMessageProvider.java>) API to customize these messages. Any syntax errors can be retrieved from the Resource using the common EMF API: the

`Resource.getErrors()` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/resource/Resource.java) and `Resource.getWarnings()` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/resource/Resource.java) method invocations.

Linker: Cross-reference Validation

Any broken cross-references can be checked generically. As cross-reference resolution is done lazily (see linking), any broken links are resolved lazily as well. If you want to validate whether all links are valid, you will have to navigate through the model so that all installed EMF proxies get resolved. This is done automatically in the editor.

Similarly to syntax errors, any unresolvable cross-links will be reported and can be obtained through the `Resource.getErrors()` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/resource/Resource.java) and `Resource.getWarnings()` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/resource/Resource.java) method invocations.

Serializer: Concrete Syntax Validation

The `ISConcreteSyntaxValidator` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/validation/ISConcreteSyntaxValidator.java>) validates all constraints that are implied by a grammar. Meeting these constraints is mandatory for a model to be serialized.

Example:

1. `MyRule`:
2. `{MySubRule} "sub"? (strVal+=ID intVal+=INT)*;`

This implies several constraints:

1. Types: only instances of `MyRule` and `MySubRule` are allowed for this rule. Subtypes are prohibited, since the parser never instantiates unknown subtypes.
2. Features: In case the `MyRule` and `MySubRule` have `EStructuralFeatures` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EStructuralFeature.java) besides `strVal` and `intVal`, only `strVal` and `intVal` may have non-transient values.
3. Quantities: The following condition must be true: `strVal.size() == intVal.size()`.
4. Values: It must be possible to convert all values to valid tokens for the used terminal rules `ID` and `INT`.

The typical use case for the concrete syntax validator is validation in non-Xtext-editors that use an `XtextResource` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/XtextResource.java>). This is the case when combining GMF and Xtext, for example. Another use case is when the semantic model is modified “manually” (not by the parser) and then serialized again. Since it is very difficult for the serializer to provide meaningful error messages, the concrete syntax validator is executed by default before serialization. A textual Xtext editor itself is *not* a valid use case. Here, the parser ensures that all syntactical constraints are met. Therefore there

is no value in additionally running the concrete syntax validator.

There are some limitations to the concrete syntax validator which result from the fact that it treats the grammar as declarative, which is something the parser doesn't always do.

- Grammar rules containing assigned actions (e.g. `{MyType.myFeature=current}`) are ignored. Unassigned actions (e.g. `{MyType}`), however, are supported.
- Grammar rules that delegate to one or more rules containing assigned actions via unassigned rule calls are ignored.
- Orders within list-features cannot be validated. e.g. Rule: `(foo+=R1 foo+=R2)*` implies that `foo` is expected to contain instances of `R1` and `R2` in an alternating order.

To use concrete syntax validation you can let Guice inject an instance of `IConcreteSyntaxValidator` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/validation/IConcreteSyntaxValidator.java>) and use it directly. Furthermore, there is an adapter (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/validation/impl/ConcreteSyntaxEValidator.java>) which allows to use the concrete syntax validator as an `EValidator` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EValidator.java). You can, for example, enable it in your runtime module, by adding:

```
1. @SingletonBinding(eager = true)
2. public Class<? extends ConcreteSyntaxEValidator> bindConcreteSyntaxEValidator() {
3.     return ConcreteSyntaxEValidator.class;
4. }
```

To customize the error messages please see `IConcreteSyntaxDiagnosticProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/validation/IConcreteSyntaxDiagnosticProvider.java>) and subclass `ConcreteSyntaxDiagnosticProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/validation/impl/ConcreteSyntaxDiagnosticProvider.java>).

Custom Validation

In addition to the afore mentioned kinds of validation, which are more or less done automatically, you can specify additional constraints specific for your Ecore model. The Xtext language generator will provide you two Java classes. The first is an abstract class generated to `src-gen/` which extends the library class `AbstractDeclarativeValidator` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/validation/AbstractDeclarativeValidator.java>). This one just registers the EPackages for which this validator introduces constraints. The other class is a subclass of that abstract class and is generated to the `src/` folder in order to be edited by you. That is where you put the constraints in.

The purpose of the `AbstractDeclarativeValidator` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/validation/AbstractDeclarativeValidator.java>) is to allow you to write constraints in a declarative way - as the class name already suggests. That is instead of writing exhaustive if-else constructs or extending the generated EMF switch you just have to add the `Check` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/>

validation/Check.java) annotation to any method and it will be invoked automatically when validation takes place. Moreover you can state for what type the respective constraint method is, just by declaring a typed parameter. This also lets you avoid any type casts. In addition to the reflective invocation of validation methods the AbstractDeclarativeValidator provides a few convenient assertions.

The Check annotation has a parameter that can be used to declare when a check should run: *FAST* will run whenever a file is modified, *NORMAL* checks will run when saving the file, and *EXPENSIVE* checks will run when explicitly validating the file via the menu option. Here is an example written in Java:

```
1. public class DomainmodelValidator extends AbstractDomainmodelValidator {
2.
3.     @Check
4.     public void checkNameStartsWithCapital(Entity entity) {
5.         if (!Character.isUpperCase(entity.getName().charAt(0))) {
6.             warning("Name should start with a capital",
7.                 DomainmodelPackage.Literals.TYPE__NAME);
8.         }
9.     }
10. }
```

You can use the IResourceValidator (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/validation/IResourceValidator.java>) to validate a given resource programmatically. Example:

```
1. @Inject IResourceValidator resourceValidator;
2.
3. public void checkResource(Resource resource) {
4.     List<Issue> issues = resourceValidator.validate(resource,
5.         CheckMode.ALL, CancelIndicator.NullImpl);
6.     for (Issue issue: issues) {
7.         switch (issue.getSeverity()) {
8.             case ERROR:
9.                 System.out.println("ERROR: " + issue.getMessage());
10.                break;
11.             case WARNING:
12.                 System.out.println("WARNING: " + issue.getMessage());
13.                break;
14.             default: // do nothing
15.         }
16.     }
17. }
```

You can also implement quick fixes for individual validation errors and warnings. See the section on quick fixes (310_eclipse_support.html#quick-fixes) for details.

Linking

The linking feature allows for specification of cross-references within an Xtext grammar. The following things are needed for the linking:

1. Declaration of a cross-reference in the grammar ([301_grammarlanguage.html#cross-references](https://eclipse.dev/Xtext/documentation/301_grammarlanguage.html#cross-references)) (or at least in the Ecore model)
2. Specification of linking semantics (usually provided via the scoping API)

Lazy Linking

Xtext uses lazy linking by default and we encourage users to stick to this because it provides many advantages, one of which is improved performance in all scenarios where you don't have to load all transitively referenced resources. Furthermore it automatically solves situations where one link relies on other links (cyclic linking dependencies are not supported, though).

When parsing a given input string, say

```
ref Entity01
```

the LazyLinker (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/linking/lazy/LazyLinker.java>) first creates an EMF proxy and assigns it to the corresponding EReference (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EReference.java). In EMF a proxy is described by a URI (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.common/src/org/eclipse/emf/common/util/URI.java), which points to the real EObject (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EObject.java). In the case of lazy linking the stored URI comprises of the context information given at parse time, which is the EObject containing the cross-reference, the actual EReference, the list index (in case it's a multi-valued cross-reference) and the string which represented the cross-link in the concrete syntax. The latter usually corresponds to the name of the referenced EObject. In EMF a URI consists of information about the resource the EObject is contained in as well as a so called fragment part, which is used to find the EObject within that resource. When an EMF proxy is resolved, the current ResourceSet (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/resource/ResourceSet.java) is asked. The resource set uses the first part to obtain (i.e. load if it is not already loaded) the resource. Then the resource is asked to return the EObject based on the fragment in the URI. The actual cross-reference resolution is done by LazyLinkingResource.getEObject(String) (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/linking/lazy/LazyLinkingResource.java>) which receives the fragment and delegates to the implementation of the ILinkingService (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/linking/ILinkingService.java>). The default implementation in turn delegates to the scoping API.

A simple implementation (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/linking/impl/DefaultLinkingService.java>) of the linking service is shipped with Xtext and used for any grammar as default. Usually any necessary customization of the linking behavior can best be described using the scoping API.

Scoping

Using the scoping API one defines which elements are referable by a given reference. For instance,

using the entities example a feature contains a cross-reference to a type:

```
1. datatype String
2.
3. entity HasAuthor {
4.     author: String
5. }
```

The grammar rule for features looks like this:

```
1. Feature:
2.     (many?='many')? name=ID ':' type=[Type];
```

The grammar declares that for the reference *type* only instances of the type *Type* are allowed. However, this simple declaration doesn't say anything about where to find the type. That is the duty of scopes.

An `IScopeProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/IScopeProvider.java>) is responsible for providing an `IScope` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/IScope.java>) for a given context `EObject` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EObject.java) and `EReference` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EReference.java). The returned `IScope` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/IScope.java>) should contain all target candidates for the given object and cross-reference.

```
1. public interface IScopeProvider {
2.
3.     /**
4.      * Returns a scope for the given context. The scope provides access to the
5.      * compatible
6.      * visible EObjects for a given reference.
7.      * @param context the element from which an element shall be referenced. I
8.      * t doesn't need to be the element
9.      * containing the reference, it is just used to find the mo
10.     st inner scope for given {@link EReference}.
11.     * @param reference the reference for which to get the scope.
12.     * @return {@link IScope} representing the innermost {@link IScope} for th
13.     e
14.     * passed context and reference. Note for implementors: The result
15.     * may not be <code>null</code>.
16.     * Return <code>IScope.NULLSCOPE</code> instead.
17.     */
18.     IScope getScope(EObject context, EReference reference);
19. }
```

A single `IScope` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/IScope.java>) represents an element of a linked list of scopes. That means that a scope can be nested within an outer scope. Each scope works like a symbol table or a map where the keys

are strings and the values are so-called `IEObjectDescription` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IEObjectDescription.java>), which is effectively an abstract description of a real `EObject`. In order to create `IEObjectDescriptions` for your model elements, the class `Scopes` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/Scopes.java>) is very useful.

To have a concrete example, let's deal with the following simple grammar.

```

1. grammar org.xtext.example.mydsl.MyScopingDsl with
2.         org.eclipse.xtext.common.Terminals
3.
4. generate myDsl "http://www.xtext.org/example/mydsl/MyScopingDsl"
5.
6. Root:
7.     elements+=Element;
8.
9. Element:
10.     'element' name=ID ('extends' superElement=[Element])?;
11.

```

If you want to define the scope for the *superElement* cross-reference, the following Java code is one way to go.

```

1. @Override
2. public IScope getScope(EObject context, EReference reference) {
3.     // We want to define the Scope for the Element's superElement cross-reference
4.     if (context instanceof Element
5.         && reference == MyDslPackage.Literals.ELEMENT__SUPER_ELEMENT) {
6.         // Collect a list of candidates by going through the model
7.         // EcoreUtil2 provides useful functionality to do that
8.         // For example searching for all elements within the root Object's tree
9.         EObject rootElement = EcoreUtil2.getRootContainer(context);
10.        List<Element> candidates = EcoreUtil2.getAllContentsOfType(rootElement, El
11.            ement.class);
12.        // Create IEObjectDescriptions and puts them into an IScope instance
13.        return Scopes.scopeFor(candidates);
14.    }
15.    return super.getScope(context, reference);
16. }

```

There are different useful implementations for `IScope` shipped with Xtext. We want to mention only some of them here.

The `MapBasedScope` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/impl/MapBasedScope.java>) comes with the efficiency of a map to look up a certain name. If you prefer to deal with Multimaps the `MultimapBasedScope` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/impl/MultimapBasedScope.java>) should work for you. For situations where some elements should be filtered out of an existing scope, the `FilteringScope` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/impl/FilteringScope.java>) is the right way to go. As scopes can be nested, we strongly recommend to use `FilteringScope` only for leaf scopes without nested scopes.

Coming back to our example, one possible scenario for the `FilteringScope` could be to exclude the context element from the list of candidates as it should not be a super-element of itself.

```

1.      @Override
2.      public IScope getScope(EObject context, EReference reference) {
3.          if (context instanceof Element
4.              && reference == MyDslPackage.Literals.ELEMENT__SUPER_ELEMENT) {
5.              EObject rootElement = EcoreUtil2.getRootContainer(context);
6.              List<Element> candidates = EcoreUtil2.getAllContentsOfType(rootElemen
            t, Element.class);
7.              IScope existingScope = Scopes.scopeFor(candidates);
8.              // Scope that filters out the context element from the candidates list
9.              return new FilteringScope(existingScope, (e) -> !Objects.equal(e.getEO
            bjectOrProxy(), context));
10.         }
11.         return super.getScope(context, reference);
12.     }

```

Global Scopes and Resource Descriptions

In the simple scoping example above we don't have references across model files. Neither is there a concept like a namespace which would make scoping a bit more complicated. Basically, every *Element* declared in the same resource is visible by its name. However, in the real world things are most likely not that simple: What if you want to reuse certain declared elements across different files and you want to share those as library between different users? You would want to introduce some kind of cross-resource reference.

Defining what is visible from outside the current resource is the responsibility of global scopes. As the name suggests, global scopes are provided by instances of the `IGlobalScopeProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/IGlobalScopeProvider.java>). The data structures (called index) used to store its elements are described in the next section.

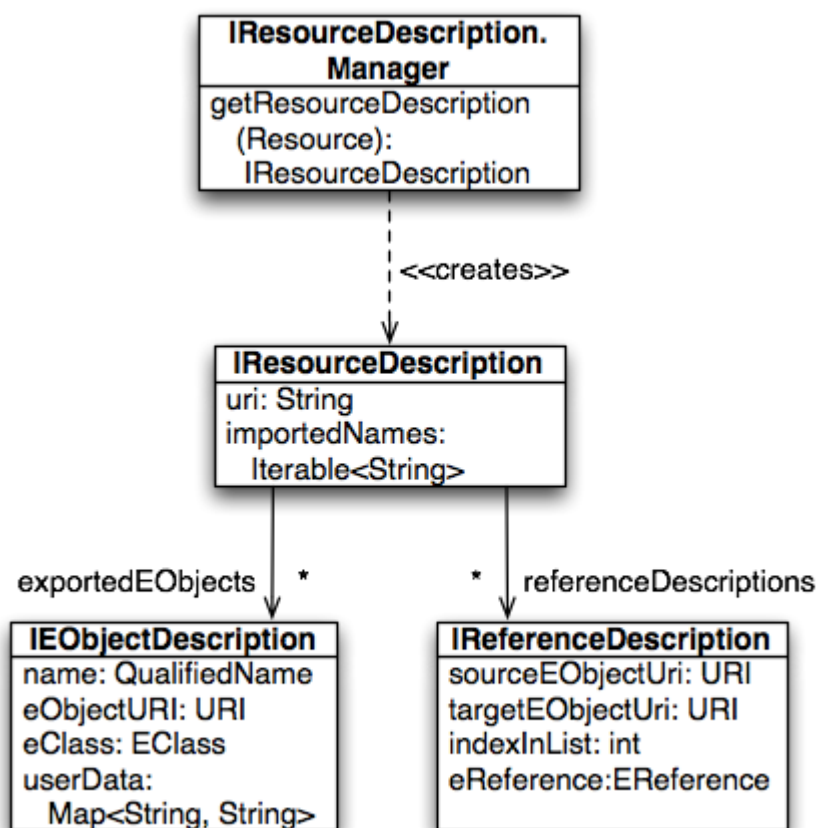
Resource andEObject Descriptions

In order to make elements of one file referable from another file, you need to export them as part of a so called `IResourceDescription` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceDescription.java>).

An `IResourceDescription` contains information about the resource itself, which primarily its URI (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.common/src/org/eclipse/emf/common/util/URI.java), a list of exported `EObjects` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/.ecore/EObject.java) in the form of `IEObjectDescription` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IEObjectDescription.java>), as well as information about outgoing cross-references and qualified names it references. The cross-references contain only resolved references, while the list of imported qualified names also contains the names that couldn't be resolved. This information is leveraged by the indexing infrastructure of Xtext in order to compute the transitive hull of

dependent resources.

For users, and especially in the context of scoping, the most important information is the list of exported EObjects. An IObjectDescription stores the URI of the actual EObject, its QualifiedName (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/naming/QualifiedName.java>), as well as its EClass (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EClass.java). In addition one can export arbitrary information using the *user data* map. The following diagram gives an overview on the description classes and their relationships.



A language is configured with default implementations of `IResourceDescription.Manager` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceDescription.java>) and `DefaultResourceDescriptionStrategy` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/impl/DefaultResourceDescriptionStrategy.java>), which are responsible to compute the list of exported `IObjectDescription`s (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IObjectDescription.java>). The Manager iterates over the whole EMF model for each Resource (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/resource/Resource.java) and asks the `ResourceDescriptionStrategy` to compute an `IObjectDescription` for each EObject. The `ResourceDescriptionStrategy` applies the `getQualifiedName(EObject obj)` from `IQualifiedNameProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/naming/IQualifiedNameProvider.java>) on the object, and if it has a qualified name an `IObjectDescription` is created and passed back to the Manager which adds it to the list of exported objects. If an EObject doesn't have a qualified name, the element is considered to be not

referable from outside the resource and consequently not indexed. If you don't like this behavior, you can implement and bind your own implementation of `IDefaultResourceDescriptionStrategy` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IDefaultResourceDescriptionStrategy.java>).

There are two different default implementations of `IQualifiedNameProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/naming/IQualifiedNameProvider.java>). Both work by looking up an `EAttribute` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+/refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EAttribute.java) 'name'. The `SimpleNameProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/naming/SimpleNameProvider.java>) simply returns the plain value, while the `DefaultDeclarativeQualifiedNameProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/naming/DefaultDeclarativeQualifiedNameProvider.java>) concatenates the simple name with the qualified name of its parent exported `EObject`. This effectively simulates the qualified name computation of most namespace-based languages such as Java. It also allows to override the name computation declaratively: Just add methods named `qualifiedName` in a subclass and give each of them one argument with the type of element you wish to compute a name for.

As already mentioned, the default implementation strategy exports every model element that the `IQualifiedNameProvider` can provide a name for. This is a good starting point, but when your models become bigger and you have a lot of them the index will become larger and larger. In most scenarios only a small part of your model should be visible from outside, hence only that small part needs to be in the index. In order to do this, bind a custom implementation of `IDefaultResourceDescriptionStrategy` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IDefaultResourceDescriptionStrategy.java>) and create index representations only for those elements that you want to reference from outside the resource they are contained in. From within the resource, references to those filtered elements are still possible as long as they have a name.

Beside the exported elements the index contains `IRReferenceDescription` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IRReferenceDescription.java>)s that contain the information who is referencing who. They are created through the `IRResourceDescription.Manager` and `IDefaultResourceDescriptionStrategy`, too. If there is a model element that references another model element, the `IDefaultResourceDescriptionStrategy` creates an `IRReferenceDescription` that contains the URI of the referencing element (*sourceEObjectURI*) and the referenced element (*targetEObjectURI*). These `IRReferenceDescriptions` are very useful to find references and calculate affected resources.

As mentioned above, in order to compute an `IRResourceDescription` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IRResourceDescription.java>) for a resource the framework asks the `IRResourceDescription.Manager` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IRResourceDescription.java>) which delegates to the `IDefaultResourceDescriptionStrategy`. To convert between a `QualifiedName` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/naming/QualifiedName.java>) and its String representation you can use the `IQualifiedNameConverter` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/naming/IQualifiedNameConverter.java>).

IQualifiedNameConverter.java). Here is some Xtend code showing how to do that:

```
1. @Inject IResourceServiceProvider.Registry rspr
2. @Inject IQualifiedNameConverter converter
3.
4. def void printExportedObjects(Resource resource) {
5.     val resServiceProvider = rspr.getResourceServiceProvider(resource.URI)
6.     val manager = resServiceProvider.getResourceDescriptionManager()
7.     val description = manager.getResourceDescription(resource)
8.     for (eod : description.exportedObjects) {
9.         println(converter.toString(eod.qualifiedName))
10.    }
11. }
```

In order to obtain a Manager (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceDescription.java>) it is best to ask the corresponding IResourceServiceProvider (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceServiceProvider.java>) as shown above. That is because each language might have a totally different implementation, and as you might refer from one language to a different language you cannot reuse the Manager (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceDescription.java>) of the first language.

Now that we know how to export elements to be referable from other resources, we need to learn how those exported IObjectDescriptions (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IObjectDescription.java>) can be made available to the referencing resources. That is the responsibility of global scoping (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/IGlobalScopeProvider.java>) which is described in the following section.

If you would like to see what's in the index, you could use the 'Open Model Element' dialog from the navigation menu entry.

Global Scopes Based On External Configuration

Instead of explicitly referring to imported resources, another option is to have some kind of external configuration in order to define what is visible from outside a resource. Java for instance uses the notion of the class path to define containers (jars and class folders) which contain referenceable elements. In the case of Java the order of such entries is also important.

To enable support for this kind of global scoping in Xtext, a DefaultGlobalScopeProvider (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/impl/DefaultGlobalScopeProvider.java>) has to be bound to the IGlobalScopeProvider (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/IGlobalScopeProvider.java>) interface. By default Xtext leverages the class path mechanism since it is well designed and already understood by most of our users. The available tooling provided by JDT and PDE to configure the class path adds even more value. However, it is just a default: you can reuse the infrastructure without using Java and be independent of the JDT.

In order to know what is available in the "world", a global scope provider which relies on external configuration needs to read that configuration in and be able to find all candidates for a certain

EReference (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+/refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EReference.java). If you don't want to force users to have a folder and file name structure reflecting the actual qualified names of the referenceable EObjects (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+/refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EObject.java), you'll have to load all resources up front and either keep holding them in memory or remember all information which is needed for the resolution of cross-references. In Xtext that information is provided by a so-called IObjectDescription (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IObjectDescription.java>).

About the Index, Containers and Their Manager

Xtext ships with an index which remembers all IResourceDescription (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceDescription.java>) and their IObjectDescription (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IObjectDescription.java>) objects. In the IDE-context (i.e. when running the editor, etc.) the index is updated by an incremental project builder. As opposed to that, in a non-UI context you typically do not have to deal with changes, hence the infrastructure can be much simpler. In both situations the global index state is held by an implementation of IResourceDescriptions (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceDescriptions.java>) (note the plural form!). The bound singleton in the UI scenario is even aware of unsaved editor changes, such that all linking happens to the latest possibly unsaved version of the resources. You will find the Guice configuration of the global index in the UI scenario in SharedModule (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.ui.shared/src/org/eclipse/xtext/ui/shared/internal/SharedModule.java>).

The index is basically a flat list of instances of IResourceDescription (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceDescription.java>). The index itself doesn't know about visibility constraints due to class path restriction. Rather than that, they are defined by the referencing language by means of so called IContainers (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IContainer.java>): While Java might load a resource via ClassLoader.loadResource() (<https://docs.oracle.com/javase/11/docs/api/java/lang/ClassLoader.html>) (i.e. using the class path mechanism), another language could load the same resource using the file system paths.

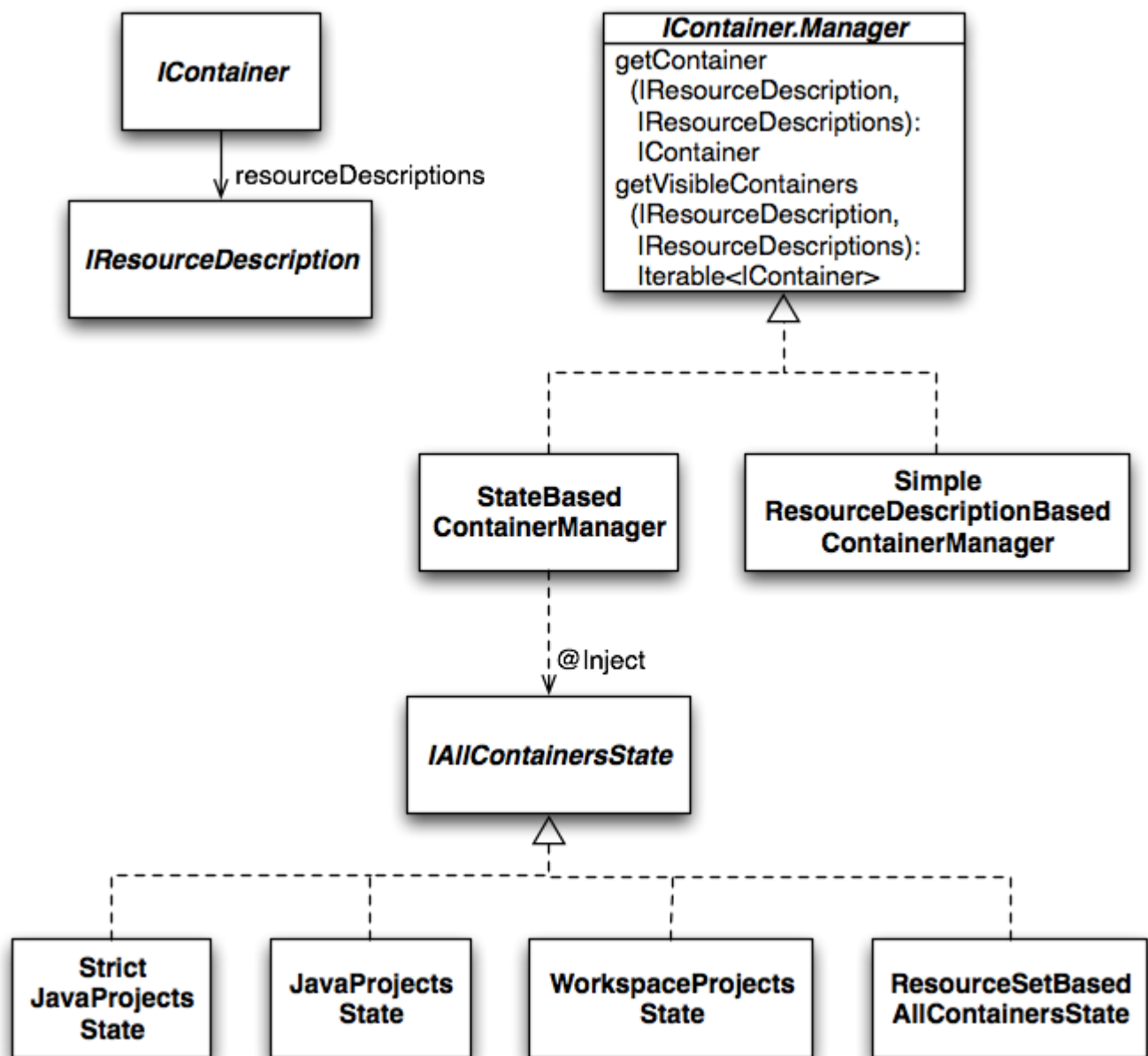
Consequently, the information which container a resource belongs to depends on the referencing context. Therefore an IResourceServiceProvider (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceServiceProvider.java>) provides another interesting service, which is called IContainer.Manager (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IContainer.java>). For a given IResourceDescription (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceDescription.java>), the Manager provides you the IContainer (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IContainer.java>) as well as a list of all IContainers which are visible from there. Note that the index (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceDescriptions.java>) is globally shared between all languages while the Manager, which adds the semantics of containers, can be very different depending on the language. The following method

lists all resources visible from a given Resource (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/resource/Resource.java):

```
1. @Inject IContainer.Manager manager
2.
3. def void printVisibleResources(Resource resource, IResourceDescriptions index) {
4.     val descr = index.getResourceDescription(resource.URI)
5.     for (visibleContainer : manager.getVisibleContainers(descr, index)) {
6.         for (visibleResourceDesc : visibleContainer.resourceDescriptions) {
7.             println(visibleResourceDesc.URI)
8.         }
9.     }
10. }
```

Xtext ships two implementations of `IContainer.Manager` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IContainer.java>) which are bound with Guice: The default binding is to `SimpleResourceDescriptionsBasedContainerManager` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/impl/SimpleResourceDescriptionsBasedContainerManager.java>), which assumes all `IResourceDescriptions` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IResourceDescription.java>) to be in a single common container. If you don't care about container support, you'll be fine with this one. Alternatively, you can bind `StateBasedContainerManager` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/containers/StateBasedContainerManager.java>) and an additional `IAllContainersState` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/containers/IAllContainersState.java>) which keeps track of the set of available containers and their visibility relationships.

Xtext offers a couple of strategies for managing containers: If you're running an Eclipse workbench, you can define containers based on Java projects and their class paths or based on plain Eclipse projects. Outside Eclipse, you can provide a set of file system paths to be scanned for models. All of these only differ in the bound instance of `IAllContainersState` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/containers/IAllContainersState.java>) of the referring language. These will be described in detail in the following sections.



JDT-Based Container Manager

As JDT is an Eclipse feature, this JDT-based container management is only available in the UI scenario. It assumes so called `IPackageFragmentRoots` (<https://help.eclipse.org/luna/topic/org.eclipse.jdt.doc.isv/reference/api/org/eclipse/jdt/core/IPackageFragmentRoot.html>) as containers. An `IPackageFragmentRoot` in JDT is the root of a tree of Java model elements. It usually refers to

- a source folder of a Java project,
- a referenced jar,
- a class path entry of a referenced Java project, or
- the exported packages of a required PDE plug-in.

So for an element to be referable, its resource must be on the class path of the caller's Java project and it must be exported (as described above).

As this strategy allows to reuse a lot of nice Java things like jars, OSGi, maven, etc. it is part of the default: You should not have to reconfigure anything to make it work. Nevertheless, if you messed something up, make sure you bind

```

1. public Class<? extends IContainer.Manager> bindIContainer$Manager() {
2.     return StateBasedContainerManager.class;
3. }

```

in the runtime module and

```

1. public Provider<IAllContainersState> provideIAllContainersState() {
2.     return org.eclipse.xtext.ui.shared.Access.getJavaProjectsState();
3. }

```

in the Eclipse UI module of the referencing language. The latter looks a bit more difficult than a common binding, as we have to bind a global singleton to a Guice provider. A `StrictJavaProjectsState` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.ui/src/org/eclipse/xtext/ui/containers/StrictJavaProjectsState.java>) requires all elements to be on the class path, while the default `JavaProjectsState` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.ui/src/org/eclipse/xtext/ui/containers/JavaProjectsState.java>) also allows models in non-source folders.

Eclipse Project-Based Containers

If the class path based mechanism doesn't work for your case, Xtext offers an alternative container manager based on plain Eclipse projects: Each project acts as a container and the project references (*Properties* → *Project References*) are the visible containers.

In this case, your runtime module should use the `StateBasedContainerManager` as shown above and the Eclipse UI module should bind

```

1. public Provider<IAllContainersState> provideIAllContainersState() {
2.     return org.eclipse.xtext.ui.shared.Access.getWorkspaceProjectsState();
3. }

```

ResourceSet-Based Containers

If you need an `IContainer.Manager` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IContainer.java>) that is independent of Eclipse projects, you can use the `ResourceSetBasedAllContainersState` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/containers/ResourceSetBasedAllContainersState.java>). This one can be configured with a mapping of container handles to resource URIs.

Local Scoping

We now know how the outer world of referenceable elements can be defined in Xtext. Nevertheless, not everything is available in all contexts and with a global name. Rather than that, each context can usually have a different scope. As already stated, scopes can be nested, i.e. a scope can contain elements of a parent scope in addition to its own elements. When parent and child scope contain different elements with the same name, the parent scope's element will usually be *shadowed* by the element from the child scope.

To illustrate that, let's have a look at Java: Java defines multiple kinds of scopes (object scope, type scope, etc.). For Java one would create the scope hierarchy as commented in the following example:


```

1. // file contents scope
2. import static my.Constants.STATIC;
3.
4. public class ScopeExample { // class body scope
5.     private Object field = STATIC;
6.
7.     private void method(String param) { // method body scope
8.         String localVar = "bar";
9.         innerBlock: { // block scope
10.             String innerScopeVar = "foo";
11.             Object field = innerScopeVar;
12.             // the scope hierarchy at this point would look like this:
13.             //   blockScope{field,innerScopeVar}->
14.             //   methodScope{localVar, param}->
15.             //   classScope{field}-> ('field' is shadowed)
16.             //   fileScope{STATIC}->
17.             //   classpathScope{
18.             //       'all qualified names of accessible static fields'} ->
19.             //   NULLSCOPE{}
20.             //
21.         }
22.         field.add(localVar);
23.     }
24. }

```

In fact the class path scope should also reflect the order of class path entries. For instance:

```

1. classpathScope{stuff from bin/}
2. -> classpathScope{stuff from foo.jar/}
3. -> ...
4. -> classpathScope{stuff from JRE System Library}
5. -> NULLSCOPE{}

```

Please find the motivation behind this and some additional details in this blog post (<http://blog.efftinge.de/2009/01/xtext-scopes-and-emf-index.html>).

Imported Namespace Aware Scoping

The imported namespace aware scoping is based on qualified names and namespaces. It adds namespace support to your language, which is comparable and similar to namespaces in Scala and C#. Scala and C# both allow to have multiple nested packages within one file, and you can put imports per namespace, such that imported names are only visible within that namespace. See the domain model example: its scope provider extends `ImportedNamespaceAwareLocalScopeProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/impl/ImportedNamespaceAwareLocalScopeProvider.java>).

Importing Namespaces

The `ImportedNamespaceAwareLocalScopeProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/impl/ImportedNamespaceAwareLocalScopeProvider.java>) looks up `EAttributes` (<https://git.eclipse.org/r/>

plugins/gitiles/emf/org.eclipse.emf/+/refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EAttribute.java) with name *importedNamespace* and interprets them as import statements.

```

1. PackageDeclaration:
2.     'package' name=QualifiedName '{'
3.         (elements+=AbstractElement)*
4.     '}' ;
5.
6. AbstractElement:
7.     PackageDeclaration | Type | Import;
8.
9. QualifiedName:
10.    ID ('.' ID)*;
11.
12. Import:
13.    'import' importedNamespace=QualifiedNameWithWildcard;
14.
15. QualifiedNameWithWildcard:
16.    QualifiedName '.*' ? ;

```

Qualified names with or without a wildcard at the end are supported. For an import of a qualified name the simple name is made available as we know from e.g. Java, where `import java.util.Set` makes it possible to refer to `java.util.Set` (<https://docs.oracle.com/javase/11/docs/api/java/util/Set.html>) by its simple name `Set`. Contrary to Java, the import is not active for the whole file, but only for the namespace it is declared in and its child namespaces. That is why you can write the following in the example DSL:

```

1. package foo {
2.     import bar.Foo
3.     entity Bar extends Foo {
4.     }
5. }
6.
7. package bar {
8.     entity Foo {}
9. }

```

Of course the declared elements within a package are as well referable by their simple name:

```

1. package bar {
2.     entity Bar extends Foo {}
3.     entity Foo {}
4. }

```

Value Converter

Value converters are registered to convert the parsed text into a data type instance and vice versa. The primary hook is the `IValueConverterService` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/conversion/IValueConverterService.java>) and the concrete

implementation can be registered via the runtime Guice module (302_configuration.html#guicemodules). Simply override the corresponding binding in your runtime module like shown in this example:

```
1. public Class<? extends IValueConverterService> bindIValueConverterService() {  
2.     return MySpecialValueConverterService.class;  
3. }
```

The easiest way to register additional value converters is to make use of `AbstractDeclarativeValueConverterService` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/conversion/impl/AbstractDeclarativeValueConverterService.java>), which allows to declaratively register an `IValueConverter` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/conversion/IValueConverter.java>) by means of an annotated method.

```
1. @ValueConverter(rule = "MyRuleName")  
2. public IValueConverter<MyDataType> getMyRuleNameConverter() {  
3.     return new MyValueConverterImplementation();  
4. }
```

If you use the common terminals grammar `org.eclipse.xtext.common.Terminals` you should extend the `DefaultTerminalConverters` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/common/services/DefaultTerminalConverters.java>) and override or add value converters by adding the respective methods. In addition to the explicitly defined converters in the default implementation, a delegating converter is registered for each available `EDataType` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+/refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EDataType.java). The delegating converter reuses the functionality of the corresponding EMF `EFactory` (https://git.eclipse.org/r/plugins/gitiles/emf/org.eclipse.emf/+/refs/tags/R2_20_0/plugins/org.eclipse.emf.ecore/src/org/eclipse/emf/ecore/EFactory.java).

Many languages introduce a concept for qualified names, i.e. names composed of namespaces separated by a delimiter. Since this is such a common use case, Xtext provides an extensible converter implementation for qualified names. The `QualifiedNameValueConverter` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/conversion/impl/QualifiedNameValueConverter.java>) handles comments and white spaces gracefully and is capable to use the appropriate value converter for each segment of a qualified name. This allows for individually quoted segments. The domainmodel example shows how to use it.

The protocol of an `IValueConverter` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/conversion/IValueConverter.java>) allows to throw a `ValueConverterException` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/conversion/ValueConverterException.java>) if something went wrong. The exception is propagated as a syntax error by the parser or as a validation problem by the `ConcreteSyntaxValidator` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/validation/impl/ConcreteSyntaxValidator.java>) if the value cannot be converted to a valid string. The `AbstractLexerBasedConverter` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/conversion/impl/AbstractLexerBasedConverter.java>) is useful

when implementing a custom value converter. If the converter needs to know about the rule that it currently works with, it may implement the interface `RuleSpecific` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/conversion/IValueConverter.java>). The framework will set the rule such that the implementation can use it afterwards.

Serialization

Serialization is the process of transforming an EMF model into its textual representation. Thereby, serialization complements parsing and lexing.

In Xtext, the process of serialization is split into the following steps:

1. Validating the semantic model. This is optional, enabled by default, done by the concrete syntax validator and can be turned off in the save options.
2. Matching the model elements with the grammar rules and creating a stream of tokens. This is done by the parse tree constructor.
3. Associating comments with semantic objects. This is done by the comment associator.
4. Associating existing nodes from the node model with tokens from the token stream.
5. Merging existing white space and line-wraps into the token stream.
6. Adding further needed white space or replacing all white space using a formatter.

Serialization is invoked when calling `XtextResource.save(..)` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/XtextResource.java>). Furthermore, the `Serializer` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parsetree/reconstr/Serializer.java>) provides resource-independent support for serialization. Another situation that triggers serialization is applying quick fixes ([310_eclipse_support.html#quick-fixes](https://eclipse.dev/eclipse4/eclipse_support.html#quick-fixes)) with semantic modifications. Serialization is *not* called when a textual editors contents is saved to disk.

The Contract

The contract of serialization says that a model which is saved (serialized) to its textual representation and then loaded (parsed) again yields a new model that is equal to the original model. Please be aware that this does *not* imply that loading a textual representation and serializing it back produces identical textual representations. However, the serialization algorithm tries to restore as much information as possible. That is, if the parsed model was not modified in-memory, the serialized output will usually be equal to the previous input. Unfortunately, this cannot be ensured for each and every case. A use case where it is hardly possible, is shown in the following example:

1. `MyRule:`
2. `(xval+=ID | yval+=INT)*;`

The given *MyRule* reads *ID*- and *INT*-elements which may occur in an arbitrary order in the textual representation. However, when serializing the model all *ID*-elements will be written first and then all *INT*-elements. If the order is important it can be preserved by storing all elements in the same list - which may require wrapping the *ID*- and *INT*-elements into other objects.

Roles of the Semantic Model and the Node Model During Serialization

A serialized document represents the state of the semantic model. However, if there is a node model available (i.e. the semantic model has been created by the parser), the serializer

- preserves existing white spaces from the node model.
- preserves existing comments from the node model.
- preserves the representation of cross-references: If a cross-referenced object can be identified by multiple names (i.e. `scoping` returns multiple `EObjectDescriptions` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/EObjectDescription.java>) for the same object), the serializer tries to keep the name that was used in the input file.
- preserves the representation of values: For values handled by the value converter, the serializer checks whether the textual representation converted to a value equals the value from the semantic model. If that is true, the textual representation is kept.

Parse Tree Constructor

The parse tree constructor usually does not need to be customized since it is automatically derived from the Xtext Grammar (301_grammarlanguage.html). However, it can be helpful to look into it to understand its error messages and its runtime performance.

For serialization to succeed, the parse tree constructor must be able to *consume* every non-transient element of the to-be-serialized EMF model. To *consume* means, in this context, to write the element to the textual representation of the model. This can turn out to be a not-so-easy-to-fulfill requirement, since a grammar usually introduces implicit constraints to the EMF model as explained for the concrete syntax validator.

If a model can not be serialized, an `XtextSerializationException` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parsetre/reconstr/XtextSerializationException.java>) is thrown. Possible reasons are listed below:

- A model element can not be consumed. This can have the following reasons/solutions:
 - The model element should not be stored in the model.
 - The grammar needs an assignment which would consume the model element.
 - The transient value service can be used to indicate that this model element should not be consumed.
- An assignment in the grammar has no corresponding model element. The default transient value service considers a model element to be transient if it is *unset* or *equals* its default value. However, the parse tree constructor may serialize default values if this is required by a grammar constraint to be able to serialize another model element. The following solution may help to solve such a scenario:
 - A model element should be added to the model.
 - The assignment in the grammar should be made optional.
- The type of the model element differs from the type in the grammar. The type of the model element must be identical to the return type of the grammar rule or the action's type. Subtypes are not allowed.
- Value conversion fails. The value converter can indicate that a value is not serializable by throwing a `ValueConverterException` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/conversion/ValueConverterException.java>).

- An enum literal is not allowed at this position. This can happen if the referenced enum rule only lists a subset of the literals of the actual enumeration.

To understand error messages and performance issues of the parse tree constructor it is important to know that it implements a backtracking algorithm. This basically means that the grammar is used to specify the structure of a tree in which one path (from the root node to a leaf node) is a valid serialization of a specific model. The parse tree constructor's task is to find this path - with the condition that all model elements are consumed while walking this path. The parse tree constructor's strategy is to take the most promising branch first (the one that would consume the most model elements). If the branch leads to a dead end (for example, if a model element needs to be consumed that is not present in the model), the parse tree constructor goes back the path until a different branch can be taken. This behavior has two consequences:

- In case of an error, the parse tree constructor has found only dead ends but no leaf. It cannot tell which dead end is actually erroneous. Therefore, the error message lists dead ends of the longest paths, a fragment of their serialization and the reason why the path could not be continued at this point. The developer has to judge on his own which reason is the actual error.
- For reasons of performance, it is critical that the parse tree constructor takes the most promising branch first and detects wrong branches early. One way to achieve this is to avoid having many rules which return the same type and which are called from within the same alternative in the grammar.

Options

`SaveOptions` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/SaveOptions.java>) can be passed to `XtextResource.save(options)` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/XtextResource.java>) and to `Serializer.serialize(..)` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parsetree/reconstr/Serializer.java>). Available options are:

- **Formatting.** Default: `false`. If enabled, it is the formatters job to determine all white space information during serialization. If disabled, the formatter only defines white space information for the places in which no white space information can be preserved from the node model. E.g. When new model elements are inserted or there is no node model.
- **Validating.** Default: `true`: Run the concrete syntax validator before serializing the model.

Preserving Comments from the Node Model

The `ICommentAssociater` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parsetree/reconstr/ICommentAssociater.java>) associates comments with semantic objects. This is important in case an element in the semantic model is moved to a different position and the model is serialized, one expects the comments to be moved to the new position in the document as well.

Which comment belongs to which semantic object is surely a very subjective issue. The default implementation (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parsetree/reconstr/impl/DefaultCommentAssociater.java>) behaves as follows, but can be customized:

- If there is a semantic token before a comment and in the same line, the comment is associated with this token's semantic object.
- In all other cases, the comment is associated with the semantic object of the next following object.

Transient Values

Transient values are values or model elements which are not persisted (written to the textual representation in the serialization phase). If a model contains model elements which can not be serialized with the current grammar, it is critical to mark them transient using the `ITransientValueService` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parsetree/reconstr/ITransientValueService.java>), or serialization will fail. The default implementation marks all model elements transient, which are `eStructuralFeature.isTransient()` or not `eObject.eIsSet(eStructuralFeature)`. By default, EMF returns `false` for `eIsSet(..)` if the value equals the default value.

Unassigned Text

If there are calls of data type rules or terminal rules that do not reside in an assignment, the serializer by default doesn't know which value to use for serialization.

Example:

```
1. PluralRule:
2.   'contents:' count=INT PLURAL;
3.
4. terminal PLURAL:
5.   'item' | 'items';
```

Valid models for this example are `contents 1 item` or `contents 5 items`. However, it is not stored in the semantic model whether the keyword *item* or *items* has been parsed. This is due to the fact that the rule call `PLURAL` is unassigned. However, the parse tree constructor needs to decide which value to write during serialization. This decision can be made by customizing the `IValueSerializer.serializeUnassignedValue(EObject, RuleCall, INode)` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parsetree/reconstr/ITokenSerializer.java>).

Cross-Reference Serializer

The cross-reference serializer specifies which values are to be written to the textual representation for cross-references. This behavior can be customized by implementing `ICrossReferenceSerializer` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parsetree/reconstr/ITokenSerializer.java>). The default implementation delegates to various other services such as the `IScopeProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/scoping/IScopeProvider.java>) or the `LinkingHelper` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/linking/impl/LinkingHelper.java>) each of

which may be the better place for customization.

Merge White Space

After the parse tree constructor has done its job to create a stream of tokens which are to be written to the textual representation, and the comment associator has done its work, the existing white spaces from the node model are merged into the stream.

The strategy is as follows: If two tokens follow each other in the stream and the corresponding nodes in the node model follow each other as well, then the white space information in between is kept. In all other cases it is up to the formatter to calculate new white space information.

Token Stream

The parse tree constructor and the formatter use an `ITokenStream` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parsetree/reconstr/ITokenStream.java>) for their output, and the latter for its input as well. This allows for chaining the two components. Token streams can be converted to a `String` (<https://docs.oracle.com/javase/11/docs/api/java/lang/String.html>) using the `TokenStringBuffer` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parsetree/reconstr/impl/TokenStringBuffer.java>) and to a `Writer` (<https://docs.oracle.com/javase/11/docs/api/java/io/Writer.html>) using the `WriterTokenStream` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parsetree/reconstr/impl/WriterTokenStream.java>).

```
1. public interface ITokenStream {  
2.  
3.     void flush() throws IOException;  
4.     void writeHidden(EObject grammarElement, String value);  
5.     void writeSemantic(EObject grammarElement, String value);  
6. }
```

Tree Manipulation

An implementation of `IDerivedStateComputer` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/IDerivedStateComputer.java>) can be used to modify the objects in the abstract syntax tree after parsing. Commonly, implementations of this interface are used to modify the objects' features programmatically.

As an example, let's take a simple Entity DSL defined by the following grammar:


```
1. grammar org.xtext.example.mydsl.MyDsl with org.eclipse.xtext.common.Terminals
2.
3. generate myDsl "http://www.xtext.org/example/mydsl/MyDsl"
4.
5. Model:
6.     entities+=Entity*;
7.
8. Entity:
9.     'Entity' name=QUOTED_NAME shortName=ID?;
10.
11. terminal QUOTED_NAME:
12.     '\'' ID (ID | ' ')* '\'';
```

This grammar allows declaring that an Entity has got a name and a short name. The former is mandatory, and composed by one or more tokens matched by the ID terminal rule, each followed by an arbitrary number of whitespaces. The resulting name must then be wrapped in single quotation marks. The latter is optional, and consists of a simple ID token. Suppose we want all Entities in our AST to have a short name: we can use a custom IDerivedStateComputer to achieve this.

Let's see a sample implementation of this interface that takes care of setting the `shortName` feature of Entities which lack a short name. We want to set it to the name of the Entity, deprived of the quotation marks, without any whitespace, and suffixed with “_computed”. For this purpose, we write the following implementation and place it, for instance, inside the `org.xtext.example.mydsl.services` package of our DSL project:

```

1. package org.xtext.example.mydsl.services;
2.
3. import org.eclipse.xtext.resource.DerivedStateAwareResource;
4. import org.eclipse.xtext.resource.IDerivedStateComputer;
5. import org.xtext.example.mydsl.myDsl.Entity;
6.
7. public class MyDslDerivedStateComputer implements IDerivedStateComputer {
8.     private static final String SUFFIX = "_computed";
9.
10.    @Override
11.    public void installDerivedState(DerivedStateAwareResource resource, boolean pr
eLinkingPhase) {
12.        resource.getAllContents().forEachRemaining(eObject -> {
13.            if (eObject instanceof final Entity entity) {
14.                if (entity.getShortName() == null) {
15.                    final String name = entity.getName();
16.                    final String shortName = name
17.                        .substring(1, name.length() - 1)
18.                        .replaceAll("\\s", "")
19.                        .trim()
20.                        .concat(SUFFIX);
21.                    entity.setShortName(shortName);
22.                }
23.            }
24.        });
25.    }
26.
27.    @Override
28.    public void discardDerivedState(DerivedStateAwareResource resource) {
29.        resource.getAllContents().forEachRemaining(eObject -> {
30.            if (eObject instanceof final Entity entity) {
31.                if (entity.getShortName().endsWith(SUFFIX)) {
32.                    entity.setShortName(null);
33.                }
34.            }
35.        });
36.    }
37. }

```

The `installDerivedState()` method is responsible for applying the desired changes. For serialization purposes, we also need to implement `discardDerivedState()` to tell the framework how to revert the changes applied programmatically. Note that this implementation sets to `null` the `shortName` feature of any `Entity` whose short name ends with the suffix, regardless of the origin of such name (input by the user or set by `installDerivedState()`).

In order for Xtext to actually use our custom `IDerivedStateComputer`, as a last step we need to bind it in our runtime module:

```
1. public class MyDslRuntimeModule extends AbstractMyDslRuntimeModule {
2.     public Class<? extends IDerivedStateComputer> bindIDerivedStateComputer()
3.     {
4.         return MyDslDerivedStateComputer.class;
5.     }
6.     @Override
7.     public Class<? extends XtextResource> bindXtextResource() {
8.         return DerivedStateAwareResource.class;
9.     }
10.
11.     public Class<? extends IResourceDescription.Manager> bindIResourceDescript
12.     ionManager() {
13.         return DerivedStateAwareResourceDescriptionManager.class;
14.     }
```

Formatting

Formatting (aka. pretty printing) is the process of rearranging the text in a document to improve the readability without changing the semantic value of the document. Therefore a formatter is responsible for arranging line-wraps, indentation, whitespace, etc. in a text to emphasize its structure, but it is not supposed to alter a document in a way that impacts the semantic model.

The actual formatting is done by constructing a list of text replacements. A text replacement describes a new text which should replace an existing part of the document, described by offset and length. Applying the text replacements turns the unformatted document into a formatted document.

To invoke the formatter programmatically, you need to instantiate a request (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/formatting2/FormatterRequest.java>) and pass it to the formatter (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/formatting2/IFormatter2.java>). The formatter will return a list of text replacements. The document modification itself can be performed by an `ITextRegionRewriter` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/formatting2/regionaccess/ITextRegionRewriter.java>).

Implementors of a formatter should extend `AbstractFormatter2` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/formatting2/AbstractFormatter2.java>) and add dispatch methods for the model elements that should be formatted. The format routine has to be invoked recursively if the children of an object should be formatted, too.

The following example illustrates that pattern. An instance of `PackageDeclaration` is passed to the `format` method along with the current formattable document (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/formatting2/IFormattableDocument.java>). In this scenario, the package name is surrounded by a single space, the curly brace is followed by a new line and increased indentation etc. All elements within that package should be formatted, too, thus `format(..)` is invoked on these as well.

```

1. def dispatch void format(PackageDeclaration p, extension IFormattableDocument doc)
   {
2.     p.regionFor.feature(PACKAGE_DECLARATION__NAME).surround[oneSpace]
3.     interior(
4.         p.regionFor.keyword('{').append[newLine],
5.         p.regionFor.keyword('}'),
6.         [indent]
7.     )
8.     for (element : p.elements) {
9.         format(element, doc)
10.        element.append[setNewLines(1, 1, 2)]
11.    }
12. }

```

The API is designed in a way that allows to describe the formatting in a declarative way by calling methods on the `IHiddenRegionFormatter` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/formatting2/IHiddenRegionFormatter.java>) which is made available inside invocations of `prepend`, `surround` or `append` to specify the formatting rules. This can be done in arbitrary order – the infrastructure will reorder all the configurations to execute them from top to bottom of the document. If the configuration-based approach is not sufficient for a particular use case, the document (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/formatting2/IFormattableDocument.java>) also accepts imperative logic that is associated with a given range. The `ITextReplacer` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/formatting2/ITextReplacer.java>) that can be added directly to the document allows to perform all kinds of modifications to the text in the region that it is associated with.

More detailed information about the API is available as JavaDoc on the `org.eclipse.xtext.formatting2` package (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/formatting2/package-info.java>).

Character Encoding

Encoding, aka. *character set*, describes the way characters are encoded into bytes and vice versa. Famous standard encodings are *UTF-8* or *ISO-8859-1*. The list of available encodings can be determined by calling `Charset.availableCharsets()` (<https://docs.oracle.com/javase/11/docs/api/java/nio/charset/Charset.html>). There is also a list of encodings and their canonical Java names in the API docs (<https://download.oracle.com/javase/1.5.0/docs/guide/intl/encoding.doc.html>).

Unfortunately, each platform and/or spoken language tends to define its own native encoding, e.g. *Cp1258* on Windows in Vietnamese or *MacIceland* on Mac OS X in Icelandic.

In an Eclipse workspace, files, folders, projects can have individual encodings, which are stored in the hidden file `.settings/org.eclipse.core.resources.prefs` in each project. If a resource does not have an explicit encoding, it inherits the one from its parent recursively. Eclipse chooses the native platform encoding as the default for the workspace root. You can change the default workspace encoding in the Eclipse preferences *Preferences* → *Workspace* → *Default text encoding*. If you develop on different platforms, you should consider choosing an explicit common encoding for your text or code files,

especially if you use special characters.

While Eclipse allows to define and inspect the encoding of a file, your file system usually doesn't. Given an arbitrary text file there is no general strategy to tell how it was encoded. If you deploy an Eclipse project as a jar (even a plug-in), any encoding information not stored in the file itself is lost, too. Some languages define the encoding of a file explicitly, as in the first processing instruction of an XML file. Most languages don't. Others imply a fixed encoding or offer enhanced syntax for character literals, e.g. the unicode escape sequences `\uXXXX` in Java.

As Xtext is about textual modeling, it allows to tweak the encoding in various places.

Encoding at Language Design Time

The plug-ins created by the *New Xtext Project* wizard are by default encoded in the workspace standard encoding. The same holds for all files that Xtext generates in there. If you want to change that, e.g. because your grammar uses/allows special characters, you should manually set the encoding in the properties of these projects after their creation. Do this before adding special characters to your grammar or at least make sure the grammar reads correctly after the encoding change. To tell the Xtext generator to generate files in the same encoding, set the encoding property in the workflow, e.g.

```
1. component = XtextGenerator {
2.     configuration = {
3.         code = {
4.             encoding = "UTF-8"
5.         }
6.     ...
}
```

Encoding at Language Runtime

As each language could handle the encoding problem differently, Xtext offers a service here. The `IEncodingProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parser/IEncodingProvider.java>) has a single method `getEncoding(URI)` to define the encoding of the resource with the given URI. Users can implement their own strategy, but keep in mind that this is not intended to be a long running method. If the encoding is stored within the model file itself, it should be extractable in an easy way, like from the first line in an XML file. The default implementation returns the default Java character set in a standalone scenario.

In the Eclipse UI scenario, when there is a workspace, users will expect the encoding of the model files to be settable the same way as for other files in the workspace. The default implementation of the `IEncodingProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parser/IEncodingProvider.java>) in the Eclipse context therefore returns the file's workspace encoding for files in the workspace and delegates to the runtime implementation for all other resources, e.g. models in a jar or from a deployed plug-in. Keep in mind that you are going to lose the workspace encoding information as soon as you leave this workspace, e.g. deploy your project.

Unless you want to enforce a uniform encoding for all models of your language, we advise to override

the runtime service only. It is bound in the runtime module using the binding annotation `@Runtime` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/service/DispatchingProvider.java>):

```
1. @Override
2. public void configureRuntimeEncodingProvider(Binder binder) {
3.     binder.bind(IEncodingProvider.class)
4.         .annotatedWith(DispatchingProvider.Runtime.class)
5.         .to(MyEncodingProvider.class);
6. }
```

For a uniform encoding, bind the plain `IEncodingProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parser/IEncodingProvider.java>) to the same implementation in all modules. In the Eclipse UI module you can use similar code as above, but with *DispatchingProvider.Ui* instead of *Runtime*.

Encoding of an XtextResource

An `XtextResource` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/resource/XtextResource.java>) uses the `IEncodingProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext/src/org/eclipse/xtext/parser/IEncodingProvider.java>) of your language by default. You can override that by passing an option on load and save, e.g.

```
1. myXtextResource.load("#{XtextResource.OPTION_ENCODING -> "UTF-8"})
```

or

```
1. myXtextResource.save("#{XtextResource.OPTION_ENCODING -> "ISO-8859-1"})
```

Unit Testing

Automated tests are crucial for the maintainability and the quality of a software product. That is why it is strongly recommended to write unit tests for your language, too. The Xtext project wizard creates test projects for that purpose, which simplify the setup procedure for the basic language implementation as well as platform-specific integrations. It supports an option to either create your tests for JUnit 4 or JUnit 6. Depending on your choice your test layout will vary in some details.

Creating a Simple Test Class

The core of the test infrastructure for JUnit 4 is the `XtextRunner` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.testing/src/org/eclipse/xtext/testing/XtextRunner.java>) and the language specific `IInjectorProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.testing/src/org/eclipse/xtext/testing/IInjectorProvider.java>). Both have to be provided by means of class annotations. Your test cases should be annotated with `org.junit.Test` (<https://github.com/junit-team/junit4/blob/master/src/main/java/org/junit/Test.java>). A static import `org.junit.Assert` (<https://github.com/junit-team/junit4/blob/master/src/main/java/org/junit/>

Assert.java) makes your tests more readable.

```
1. import org.eclipse.xtext.testing.InjectWith
2. import org.eclipse.xtext.testing.XtextRunner
3. import org.junit.runner.RunWith
4. import org.junit.Test
5. import static org.junit.Assert.*
6. import org.example.domainmodel.DomainmodelInjectorProvider
7.
8. @InjectWith(DomainmodelInjectorProvider)
9. @RunWith(XtextRunner)
10. class ParserTest {
11.     @Test def void simple() {
12.         assertTrue(true)
13.     }
14. }
```

This configuration will make sure that you can use dependency injection in your test class, and that the global EMF registries are properly populated before and cleaned up after each test.

A test class for JUnit 6 looks quite similar. Instead of runners JUnit 6 has a notion of Extensions (<https://docs.junit.org/>). While there can only be one runner per test class for JUnit 4 there could be multiple extensions for JUnit 6. The replacement for the XtextRunner is the new InjectionExtension (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.testing/src/org/eclipse/xtext/testing/extensions/InjectionExtension.java>). Still needed is the language specific IInjectorProvider (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.testing/src/org/eclipse/xtext/testing/IInjectorProvider.java>). Instead of `org.junit.Test` you have to annotate your cases with `org.junit.jupiter.api.Test` (<https://github.com/junit-team/junit-framework/blob/main/junit-jupiter-api/src/main/java/org/junit/jupiter/api/Test.java>) and import the methods from `org.junit.jupiter.api.Assertions` (<https://github.com/junit-team/junit-framework/blob/main/junit-jupiter-api/src/main/java/org/junit/jupiter/api/Assertions.java>). A simple test class for JUnit 6 will then look like this:

```
1. import org.eclipse.xtext.testing.InjectWith
2. import org.eclipse.xtext.testing.extensions.InjectionExtension
3. import org.junit.jupiter.api.^extension.ExtendWith
4. import org.junit.jupiter.api.Test
5. import static org.junit.jupiter.api.Assertions.*
6. import org.example.domainmodel.DomainmodelInjectorProvider
7.
8. @InjectWith(DomainmodelInjectorProvider)
9. @ExtendWith(InjectionExtension)
10. class ParserTest {
11.     @Test def void simple() {
12.         assertTrue(true)
13.     }
14. }
```

Testing the Parser

The class ParseHelper (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.testing/>

`src/org/eclipse/xtext/testing/util/ParseHelper.java`) allows to parse an arbitrary string into an AST model. The AST model itself can be traversed and checked afterwards.

```
1. import org.eclipse.xtext.testing.util.ParseHelper
2. ...
3.
4. @Inject ParseHelper<Domainmodel> parser
5.
6. @Test
7. def void parseDomainmodel() {
8.     val model = parser.parse(''
9.         entity MyEntity {
10.             parent: MyEntity
11.         }
12.     '')
13.     val entity = model.elements.head as Entity
14.     assertSame(entity, entity.features.head.type)
15. }
```

Testing the Validator

Testing your validation is very simple with the `ValidationTestHelper` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.testing/src/org/eclipse/xtext/testing/validation/ValidationTestHelper.java>):

```
1. ...
2.
3. @Inject extension ParseHelper
4. @Inject extension ValidationTestHelper
5.
6. @Test
7. def void testLowercaseName() {
8.     val model = "entity foo {}".parse
9.     model.assertWarning(DomainmodelPackage.Literals.ENTITY, null,
10.         "Name should start with a capital")
11. }
```

See the various *assert* methods in that helper class to explore the testing capabilities. You can either assert that a given model has specific issues as in the example above, or assert that it has no issues.

Testing Multiple Languages

If in addition to the main language your tests require using other languages for references from/to your main language, you'll have to parse and load dependent resources into the same `ResourceSet` first for cross-reference resolution to work.

As the default generated `IInjectorProvider` (<https://github.com/eclipse-xtext/xtext/blob/main/org.eclipse.xtext.testing/src/org/eclipse/xtext/testing/IInjectorProvider.java>) of your main language (e.g. `DomainmodelInjectorProvider`) does not know about any other dependent languages, they must be initialized explicitly. The recommended pattern for this is to create a new subclass of the

generated *MyLanguageInjectorProvider* in your *.test project and make sure the dependent language is initialized properly. Then you can use this new injector provider instead of the original one in your test's *@InjectWith*:

```

1. public class MyLanguageWithDependenciesInjectorProvider extends MyLanguageInjector
   Provider {
2.     @Override
3.     protected Injector internalCreateInjector() {
4.         MyOtherLangLanguageStandaloneSetup.doSetup();
5.         return super.internalCreateInjector();
6.     }
7. }
8.
9. // @RunWith(XtextRunner.class) // JUnit 4
10. @ExtendWith(InjectionExtension.class) // JUnit 6
11. @InjectWith(MyLanguageWithDependenciesInjectorProvider.class)
12. public class YourTest {
13.     ...
14. }
```

You should not put injector creation for referenced languages in your standalone setup. Note that for the headless code generation use case, the Maven plug-in is configured with multiple setups, so usually there is no problem there.

You may also need to initialize imported Ecore models that are not generated by your Xtext language. This should be done by using an explicit *MyModelPackage.eINSTANCE.getName()*; in the *doSetup()* method of your respective language's *StandaloneSetup* class. Note that it is strongly recommended to follow this pattern instead of just using *@Before* methods in your *Test class, as due to internal technical reasons that won't work anymore as soon as you have more than just one *@Test*.

```

1. public class MyLanguageStandaloneSetup extends MyLanguageStandaloneSetupGenerated
   {
2.
3.     public static void doSetup() {
4.         if (!EPackage.Registry.INSTANCE.containsKey(MyPackage.eNS_URI))
5.             EPackage.Registry.INSTANCE.put(MyPackage.eNS_URI, MyPackage.eINSTANC
6. E);
7.         new MyLanguageStandaloneSetup().createInjectorAndDoEMFRegistration();
8.     }
9. }
```

This only applies to referencing dependencies to imported Ecore models and languages based on them which may be used in the test. The inherited dependencies from mixed-in grammars are automatically listed in the generated super class already, and nothing needs to be done for those.

The Xtext example projects (*File* → *New* → *Example* → *Xtext Examples*) contain further unit test cases, e.g. testing the formatter, serializer, compiler, ...etc. Feel free to study the corresponding *org.eclipse.xtext.example.<language>.tests* projects to get some inspirations on how to implement automated unit test cases for your Xtext-based language.

